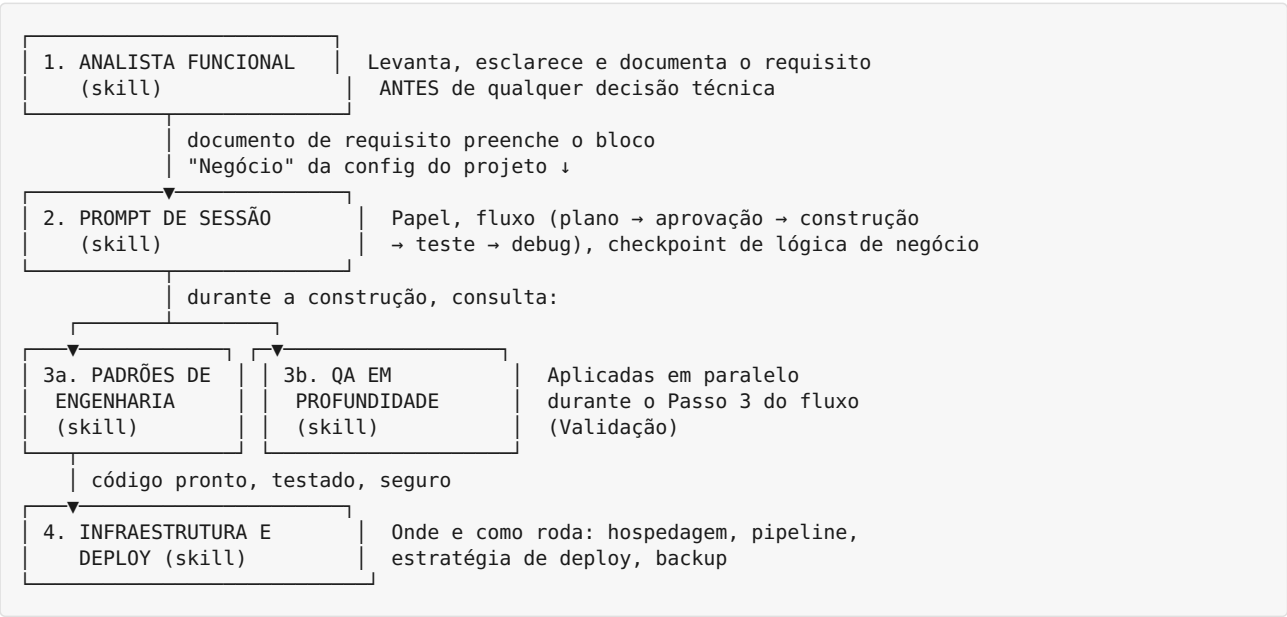


Processo de Desenvolvimento — Do Requisito ao Deploy

Este é o mapa de como as cinco peças se encaixam. Nenhuma delas cobre o que a outra já cobre — cada uma atua numa fase específica do ciclo, e o que uma produz alimenta a próxima.

No Antigravity 2.0, todas as cinco são **Skills** (o conceito de “Rule” separada foi unificado em Skill). Em versões anteriores, a peça da Etapa 2 também podia ser configurada como Rule — ambos os formatos têm o mesmo conteúdo.



Etapa 1 — Analista Funcional (`analista-funcional-requisitos`)

Quando: todo início de projeto ou feature nova, antes de qualquer linha de código ou decisão de arquitetura.

O que produz: um documento que responde por escrito: 1. Qual problema real está sendo resolvido (não a solução que foi pedida) 2. A regra de negócio em linguagem de negócio 3. As exceções à regra 4. O que acontece quando falha 5. Quem é o dono da regra

Mais: user stories com critério de aceite mensurável, requisitos não-funcionais explícitos, escopo do que fica de fora, e sign-off do dono do requisito.

Saída: esse documento preenche diretamente o bloco **Negócio** da seção `CONFIGURAÇÃO DO PROJETO` do prompt de sessão (Etapa 2). Sem essa etapa completa, a Etapa 2 vai parar e perguntar — isso é o comportamento esperado, não um obstáculo.

Etapa 2 — Papel do Engenheiro (`papel-engenheiro-senior`)

Quando: sempre ativo durante o desenvolvimento — é a skill que define como a IA se comporta como par técnico. (Em versões pré-2.0 do Antigravity, equivalente à Rule `prompt-sessao-engenheiro.md`.)

O que faz:

- Define o papel (par técnico que discorda quando necessário, não executor passivo)
- Preenche a `CONFIGURAÇÃO DO PROJETO` com os dados de negócio (vindos da Etapa 1) e técnicos
- Classifica a criticidade do que está sendo construído (CRÍTICO / IMPORTANTE / BAIXO) — isso decide o

rigor aplicado nas Etapas 3a e 3b

- Roda o fluxo de 4 passos para mudança estrutural: **Plano → Aprovação → Construção Incremental → Validação → Debug guiado**

Não repete: os critérios técnicos detalhados de segurança/performance/resiliência (isso é 3a) nem as técnicas avançadas de design de teste (isso é 3b) — apenas aciona o passo certo do fluxo no momento certo.

Etapas 3a — Padrões de Engenharia (`padroes-engenharia-software`)

Quando: durante toda a escrita/revisão de código — dispara automaticamente ao criar endpoint, integração, query, autenticação, ou antes de declarar algo pronto para produção.

O que cobre: segurança (não-negociável, sempre), performance (N+1, índice, paginação, timeout), resiliência de código (retry, idempotência, transação), observabilidade de aplicação (log estruturado, métricas), teste básico (pirâmide, cobertura mínima), e uso crítico de IA no código.

Não cobre: onde o sistema roda, pipeline de deploy, backup (isso é Etapa 4).

Etapas 3b — QA em Profundidade (`qa-sistemas-proprios`)

Quando: no Passo 3 (Validação) do fluxo da Etapa 2, quando o checklist básico de teste da Etapa 3a não é suficiente para decidir *o que* testar e com que profundidade — especialmente em lógica de negócio crítica.

O que cobre: particionamento de equivalência, valor limite, tabela de decisão, priorização de esforço por risco, teste exploratório estruturado, validação de dado externo.

Não repete: o checklist de “o que não pode faltar” da Etapa 3a — aprofunda o “como projetar bem” o teste.

Etapas 4 — Infraestrutura e Deploy (`arquitetura-infraestrutura-deploy`)

Quando: ao decidir hospedagem, montar pipeline de CI/CD, definir estratégia de deploy, ou antes de considerar o sistema “em produção” de verdade.

O que cobre: escolha de hospedagem proporcional à escala real, paridade de ambientes, infraestrutura como código, pipeline de CI/CD, estratégia de deploy com rollback definido antes (não improvisado), segredo injetado em runtime, backup com teste de restauração, custo como requisito não-funcional.

Não cobre: resiliência dentro do código (isso é 3a).

Regra geral que atravessa todas as etapas — CHA

Antes de responder ou produzir qualquer artefato em qualquer uma das quatro etapas, o critério de honestidade é:

- **Competência:** a resposta se apoia em conhecimento real ou fonte confiável — não em suposição vestida de certeza.
- **Habilidade:** só afirmar capacidade de fazer o que de fato pode ser executado e validado.
- **Atitude:** ser claro sobre o que é fato, o que é suposição a confirmar, e o que está fora do escopo de conhecimento disponível. Nunca simular certeza para parecer mais útil.

Isso já está distribuído nas seções “Uso de IA” de cada skill (nunca inventar nome de função, nunca inventar requisito, nunca aplicar mudança de infra sem revisar) — a regra geral aqui é o princípio unificador por trás de todas elas.

Resumo de uso rápido

Você está...	Use
Recebendo um pedido vago ou “preciso de um sistema que faça X”	Etapa 1
Prestes a começar a construir um módulo novo	Etapa 2, Passo 1 (Plano)
Escrevendo código de fato	Etapa 2 (fluxo) + Etapa 3a (sempre ativa)
Decidindo o que e como testar algo crítico	Etapa 3b
Preparando para subir para produção	Etapa 4
Recebendo um log de erro	Etapa 2, Passo 4 (Debug guiado)